MOBL-99: Best Practice Summary

> **Series:** MOBL — Mobile Monitoring | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 07/30/2026

Overview

This notebook consolidates every actionable best practice from the MOBL series (notebooks 01 through 12) into a single, definitive reference. Each best practice specifies the exact setting or value to use, its priority level, and which category it belongs to. Use this as a checklist when instrumenting, configuring, and operating Dynatrace native mobile monitoring.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`rum.read`, `entities.read`, `bizevents.read`, `metrics.read`
Mobile App	At least one mobile application with Dynatrace SDK integrated
Prior Knowledge	Familiarity with MOBL-01 through MOBL-12 recommended

1. SDK Setup & Configuration

Best practices for installing and configuring the Dynatrace mobile SDK across iOS, Android, and cross-platform frameworks.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Enable auto-start for production apps	iOS: `DTXAutoStart = true` in Info.plist; Android: `autoStart = true` in Gradle config	**Critical**	MOBL-02, MOBL-03
2	Enable crash reporting	iOS: `DTXCrashReportingEnabled = true`; Android: `crashReporting(true)`	**Critical**	MOBL-02, MOBL-03
3	Use Swift Package Manager over CocoaPods for iOS	Add via Xcode: File > Add Package Dependencies	**Recommended**	MOBL-02
4	Pin Gradle plugin version to a specific release	`id("com.dynatrace.instrumentation") version "8.x.x"` -- never use `latest`	**Critical**	MOBL-03
5	Use separate Application IDs for iOS and Android	Each platform gets its own Application ID and Beacon URL from Dynatrace	**Critical**	MOBL-04
6	Use separate Application IDs for prod vs staging	Configure build variant overrides in Gradle; use different Info.plist per scheme	**Recommended**	MOBL-03
7	Never commit Application IDs or Beacon URLs to public repos	Store in environment variables or `local.properties` excluded from version control	**Critical**	MOBL-03
8	Enable hybrid monitoring for WebView apps	iOS: `DTXHybridApplication = true`; Android: `hybridMonitoring(true)`	**Recommended**	MOBL-02, MOBL-03
9	For Flutter: use `dynatrace.config.yaml` at project root	Provide platform-specific `applicationId` and `beaconUrl` under `android:` and `ios:` keys	**Critical**	MOBL-04
10	For React Native: re-run `npx instrumentDynatrace` after config changes	Changing `dynatrace.config.js` does not auto-apply; the instrument command must re-run	**Critical**	MOBL-04

#	Best Practice	Recommended Setting/Value	Priority	Source
11	Verify both platform configs exist for cross-platform apps	Query `smartscapeNodes "FRONTEND" | filter frontend.type == "mobile"` and confirm both Android and iOS entries appear. Classic equivalent: `fetch dt.entity.mobile_application` -- **not** `dt.entity.device_application`, which does not exist and returns zero rows	**Critical**	MOBL-04

2. Crash Reporting & Symbolication

Best practices for ensuring crash data is complete, readable, and actionable.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Upload dSYM files for every iOS release build	Automate via Xcode Build Phase Script using `DTXDssClient`, or use Fastlane `dynatrace_process_symbols`	**Critical**	MOBL-06
2	Upload ProGuard/R8 mapping files for every Android release build	Enable in Gradle: `symbols { apitoken 'YOUR_API_TOKEN' }` for automatic upload	**Critical**	MOBL-06
3	Upload React Native source maps	Set `sourceMap: { android: true, ios: true }` in `dynatrace.config.js`	**Critical**	MOBL-04
4	Verify symbolication status after upload	Check Dynatrace UI under Mobile > Symbol Files; stack traces must show method names, not hex addresses	**Critical**	MOBL-06
5	For Bitcode iOS builds: download dSYMs from App Store Connect	Local dSYMs do not match Apple-recompiled binaries; download from Xcode Organizer	**Critical**	MOBL-06
6	Add ProGuard keep rules for Dynatrace SDK	`-keep class com.dynatrace.** { *; }` and `-dontwarn com.dynatrace.**` in `proguard-rules.pro`	**Critical**	MOBL-03
7	Report handled exceptions via SDK API	iOS: `DTXAction.reportError(withName:error:)`; Android: `Dynatrace.reportError(name, exception)`	**Recommended**	MOBL-06
8	Use descriptive error names for reported errors	`"Payment Failed"` not `"Error"` -- enables meaningful DQL grouping	**Recommended**	MOBL-06

#	Best Practice	Recommended Setting/Value	Priority	Source
9	Report errors at catch boundaries, not in utility methods	Attach full context (user action, screen name) where the error is handled	**Recommended**	MOBL-06
10	Do not report transient/expected conditions as errors	Network retries, expected timeouts, and recoverable conditions create noise	**Recommended**	MOBL-12
11	Target crash-free rate above 99.5%	< 99%: investigate immediately; 99-99.5%: review top crash groups; > 99.5%: healthy	**Critical**	MOBL-11

3. User Action Instrumentation

Best practices for tracking user interactions accurately and meaningfully.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Add manual instrumentation for SwiftUI views	Apply `.dtAction(name:)` to top-level screen views and key interaction buttons	**Critical**	MOBL-02, MOBL-05
2	Add manual instrumentation for Jetpack Compose	Use `Dynatrace.enterAction("name")` and `action.leaveAction()` for all onClick handlers	**Critical**	MOBL-03, MOBL-05
3	Always call `leaveAction()` in a finally block	Forgetting to close an action inflates duration metrics and orphans child events	**Critical**	MOBL-03, MOBL-05
4	Use descriptive, stable action names	`"Cart: Add Item"` not `"btn_click"` or `"button_42"`	**Critical**	MOBL-05
5	Never put dynamic values in action names	`"View Product Details"` not `"View Product #48291"` -- dynamic values cause cardinality explosion	**Critical**	MOBL-05
6	Never put user IDs or timestamps in action names	Creates infinite unique names, defeats grouping, and is a privacy risk	**Critical**	MOBL-05
7	Use feature-area prefixes in naming convention	`"Cart: Add Item"`, `"Search: Apply Filter"`, `"Checkout: Submit Order"`	**Recommended**	MOBL-05
8	Use consistent casing across platforms	`"Search Products"` on both iOS and Android, not mixed case variants	**Recommended**	MOBL-05
9	Avoid applying `.dtAction()` to every SwiftUI subview	Excess actions create noise; apply only to top-level screens and key interactions	**Recommended**	MOBL-02

#	Best Practice	Recommended Setting/Value	Priority	Source
10	Configure user action naming rules in Dynatrace UI	Settings > RUM > Mobile > User action naming; test rules against recent data before deploying	**Recommended**	MOBL-05
11	Keep action hierarchies shallow	Parent + 1 level of child actions maximum for nested actions	**Recommended**	MOBL-05, MOBL-12

4. Network Request Monitoring

Best practices for HTTP monitoring, backend correlation, and network performance optimization.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Instrument backend services with OneAgent or OpenTelemetry	Required for `x-dtc` header correlation; enables end-to-end distributed tracing from device to database	**Critical**	MOBL-07
2	Use manual web request tagging for non-standard transports	Add `x-dynatrace` header via `DTXAction.getRequestTag()` (iOS) or `Dynatrace.getRequestTag()` (Android) for WebSocket, gRPC, or custom HTTP clients	**Recommended**	MOBL-12
3	Enable gzip/brotli compression on API responses	Reduces response payload size by 60-80%	**Recommended**	MOBL-07
4	Implement response pagination	Avoid loading entire datasets in a single request	**Recommended**	MOBL-07
5	Use HTTP/2 or HTTP/3 for multiplexing	Reduces connection overhead and round trips	**Recommended**	MOBL-07
6	Implement client-side caching with ETags and Cache-Control	Prevents redundant network requests for unchanged resources	**Recommended**	MOBL-07
7	Serve static assets from a CDN	Reduces DNS, TCP, and TLS times for geographically distributed users	**Recommended**	MOBL-07
8	Adapt app behavior to connection type	Reduce image quality, defer non-critical requests, enable offline queueing on slow connections	**Optional**	MOBL-07
9	When TTFB is the dominant phase, investigate backend	Use frontend-to-backend correlation to trace into server-side services and database calls	**Recommended**	MOBL-07

#	Best Practice	Recommended Setting/Value	Priority	Source
10	Exclude analytics/CDN URLs from SDK monitoring	Configure `excludeURLs` patterns for `analytics.google.com`, `firebaselogging.googleapis.com`, CDN domains	**Recommended**	MOBL-12

5. Session Replay & Privacy

Best practices for configuring mobile session replay with appropriate privacy controls.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Enable session replay on crash	iOS: `DTXSessionReplayOnCrash = true`; Android: `sessionReplayOnCrash(true)`	**Critical**	MOBL-08
2	Set privacy masking to Safe as the default	iOS: `DTXSessionReplayPrivacyMode = SAFE`; Android: `sessionReplayPrivacyMode("SAFE")`	**Critical**	MOBL-08
3	Set production sample rate to 5-10%	iOS: `DTXSessionReplaySampleRate = 10`; Android: `sessionReplaySampleRate(10)`	**Critical**	MOBL-08
4	Set staging/QA sample rate to 100%	Full coverage for pre-release testing and bug verification	**Recommended**	MOBL-08
5	Use Safest masking for financial/healthcare apps	Replays show layout and navigation but no readable content	**Critical**	MOBL-08
6	Only use Custom masking when you need to unmask specific elements	Start with Safe, move to Custom only for targeted debugging; tag specific views with `dtxMaskingMode`	**Recommended**	MOBL-08
7	Temporarily increase sample rate during incidents	Raise to 50-100% when actively investigating a reported issue; lower again after resolution	**Recommended**	MOBL-08
8	Monitor DEM unit consumption for session replay	Track in Dynatrace license overview; session replay is the primary DEM cost driver	**Recommended**	MOBL-08
9	Consider shorter Grail retention for session replay data	Session replay contains more sensitive visual information than raw telemetry	**Optional**	MOBL-09

6. Session Properties & User Tagging

Best practices for enriching sessions with business context and identifying users.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Tag users with opaque IDs, never PII	`Dynatrace.identifyUser("usr_a1b2c3d4")` -- never email addresses, phone numbers, or full names	**Critical**	MOBL-09
2	Clear user tag on logout	iOS: `Dynatrace.identifyUser(nil)`; Android: `Dynatrace.identifyUser(null)`	**Critical**	MOBL-09
3	Set session properties early in the session	Call `reportValue()` immediately after login or app start so properties apply to all subsequent actions	**Recommended**	MOBL-09
4	Limit to 20-30 custom session properties per app	Excessive properties increase beacon size and processing overhead	**Recommended**	MOBL-09
5	Use enum-like string values for properties	`"tier" = "free"` not `"tier" = "Free Trial Account"` -- enables clean aggregation	**Recommended**	MOBL-09
6	Use consistent property key names across iOS and Android	Same keys on both platforms so a single DQL query covers both	**Recommended**	MOBL-09
7	Never store PII in session property values	Session properties are not subject to data masking	**Critical**	MOBL-09
8	Report A/B test variants as session properties	`reportValue("ab_test_checkout_v2", "variant_b")` -- enables correlation with performance/crash data	**Recommended**	MOBL-12

7. Privacy & Compliance

Best practices for GDPR, CCPA, and general data privacy compliance.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Implement opt-in mode for GDPR compliance	Set `DTXAutoStart = false` (iOS) or `autoStart = false` (Android); start SDK only after user consent	**Critical**	MOBL-09
2	Default data collection level to Performance until consent	`DataCollectionLevel.PERFORMANCE` captures anonymous performance data without user identification	**Critical**	MOBL-09

#	Best Practice	Recommended Setting/Value	Priority	Source
3	Persist user consent in app storage	SDK does not persist consent; store in UserDefaults (iOS) or SharedPreferences (Android) and check on each launch	**Critical**	MOBL-09
4	Provide a way to withdraw consent in app settings	Set data collection level to `Off` when user revokes consent	**Critical**	MOBL-09
5	Implement data deletion API for right-to-erasure requests	POST to `/api/v2/data-privacy/deletion` with `userId`, `startDate`, `endDate`, `dataTypes: ["RUM"]`	**Critical**	MOBL-09
6	Set shortest Grail retention period that meets business needs	Configure dedicated Grail buckets with retention matching your data minimization policy	**Recommended**	MOBL-09
7	Separate crash reporting opt-in from general monitoring	Crash reporting has its own `crashReportingOptedIn` flag independent of data collection level	**Recommended**	MOBL-09
8	Display clear, plain-language consent dialog	Explain what data is collected, why, and how; provide granular options for Performance vs User Behavior levels	**Critical**	MOBL-09
9	Consider separate Grail buckets for EU vs non-EU data	Use OpenPipeline rules to route data based on geolocation for data residency compliance	**Optional**	MOBL-09

8. Dashboards & Alerting

Best practices for operationalizing mobile monitoring with dashboards and alerts.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Include a crash-free rate single-value tile with color thresholds	Green: > 99.5%, Yellow: 98-99.5%, Red: < 98%	**Critical**	MOBL-11
2	Organize dashboard: health at top, trends in middle, drill-down at bottom	Top: single-value tiles (crash rate, sessions); Middle: timeseries (trends); Bottom: tables (top crashes, slowest actions)	**Recommended**	MOBL-11
3	Add a dashboard variable for `useraction.application`	Lets stakeholders filter to their specific app	**Recommended**	MOBL-11
4	Set default time range to 24 hours with presets for 1h, 6h, 24h, 7d	Covers most operational use cases without excessive data scanning	**Recommended**	MOBL-11

#	Best Practice	Recommended Setting/Value	Priority	Source
5	Alert on high crash rate	Static threshold: > 10 crashes in a 1-hour sliding window, severity Critical	**Critical**	MOBL-11
6	Alert on session volume drop	< 50% of rolling 7-day baseline, severity Warning	**Recommended**	MOBL-11
7	Alert on slow app launch	Average app start > 5 seconds over a 15-minute window, severity Warning	**Recommended**	MOBL-11
8	Alert on high HTTP error rate	> 5% of mobile requests returning 5xx, severity Critical	**Critical**	MOBL-11
9	Use Dynatrace Intelligence for adaptive baselines on performance metrics	Dynatrace Intelligence automatically learns patterns; add static thresholds only for hard SLA limits	**Recommended**	MOBL-11
10	Tag mobile app entities with `app-type: mobile`	Enables detected problem workflow triggers filtered to mobile-specific problems	**Recommended**	MOBL-11
11	Create audience-specific dashboards	Developers: 5-min refresh, crash detail; QA: 15-min, crash-free rate by version; Executives: daily KPI snapshot	**Recommended**	MOBL-11
12	Break down crash rate by `app.version` after every release	Identifies if a specific release introduced a regression	**Critical**	MOBL-10, MOBL-11

9. SDK Performance Optimization

Best practices for minimizing SDK overhead on battery, network, and CPU.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Exclude third-party analytics and CDN URLs from monitoring	Add `excludeURLs` patterns: `analytics.google.com/*`, `firebaselogging.googleapis.com/*`, CDN domains	**Recommended**	MOBL-12
2	Use sampling for high-traffic production apps	10-50% sampling reduces DEM consumption proportionally; even 1% of millions provides thousands of sessions	**Recommended**	MOBL-08, MOBL-12
3	Start with full monitoring, reduce only if needed	Do not prematurely optimize; measure SDK overhead first	**Recommended**	MOBL-12

#	Best Practice	Recommended Setting/Value	Priority	Source
4	Use crash-only mode for ultra-low overhead scenarios	Set performance collection level to crash-only; captures crashes and errors with minimal overhead	**Optional**	MOBL-12
5	App launch time target: under 2 seconds	< 1s: excellent; 1-2s: acceptable; 2-5s: needs improvement; > 5s: investigate immediately	**Critical**	MOBL-10, MOBL-11

10. Advanced Instrumentation

Best practices for custom business events, A/B testing, multi-app strategies, and advanced SDK usage.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use reverse-domain naming for custom business event types	`"com.myapp.purchase"` not `"purchase"` -- avoids collisions across apps and services	**Recommended**	MOBL-12
2	Send business events for conversion-critical interactions	Purchases, sign-ups, feature usage via `sendBizEvent()` with structured attributes	**Recommended**	MOBL-12
3	Report feature flag evaluations as business events	`event.type = "com.myapp.feature_flag"` with `feature.name` and `feature.variant` attributes	**Optional**	MOBL-12
4	Use nested actions for multi-step flows	Parent action wraps the full flow; child actions measure individual steps (validate, submit, confirm)	**Optional**	MOBL-12
5	For long-running actions, manage lifecycle explicitly	Call `leaveAction()` in completion handler or `finally` block; do not rely on auto-close timeout	**Critical**	MOBL-12
6	Use consistent attribute names across iOS and Android business events	Same keys on both platforms enable unified DQL queries	**Recommended**	MOBL-12
7	Never include PII in business event attributes	No email addresses, phone numbers, or personal data in custom event payloads	**Critical**	MOBL-12
8	Use consistent naming prefixes for multi-app organizations	`"MyBrand Customer"`, `"MyBrand Driver"`, `"MyBrand Admin"` for easy DQL filtering	**Recommended**	MOBL-12
9	Budget DEM units per app based on criticality	100% monitoring for critical apps; use sampling for high-traffic secondary apps	**Recommended**	MOBL-12

#	Best Practice	Recommended Setting/Value	Priority	Source
10	Attach `reportValue()` properties to actions for business context	`action.reportValue(withName: "total_price", doubleValue: 49.99)` -- becomes queryable in Grail	**Recommended**	MOBL-12

11. DQL Query Patterns

Mandatory patterns and filters for querying mobile data in Grail.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Always filter mobile data on event.provider	`filter event.provider == "www.dynatrace.com/mobile"` -- this is the key discriminator for mobile events	**Critical**	MOBL-10
2	Use `countDistinct(dt.rum.session.id)` for unique session counts	Never use `count()` for sessions -- each session generates many events	**Critical**	MOBL-10
3	Use conditional countDistinct for crash rate	`countDistinct(if(event.type == "com.dynatrace.crash", then:dt.rum.session.id, else:null))`	**Recommended**	MOBL-10
4	Add `isNotNull()` filters before grouping on optional fields	Fields like `os.type`, `device.model`, `app.version` may be null; filter first to avoid noise	**Recommended**	MOBL-10
5	Query mobile app entities for inventory	Preferred: `smartscapeNodes "FRONTEND" | filter frontend.type == "mobile"`. Classic fallback (still functional): `fetch dt.entity.mobile_application`. No time range needed; returns current state. `dt.entity.device_application` does not exist -- it returns zero rows, not an error	**Recommended**	MOBL-10
6	Filter crashes with `event.type == "com.dynatrace.crash"`	Reported errors use `event.type == "com.dynatrace.error.report"` -- distinguish between the two	**Critical**	MOBL-06, MOBL-10
7	Always specify explicit time ranges on bizevents queries	`fetch bizevents, from:-1h` or `from:-24h` -- never rely on the default 2-hour window	**Critical**	MOBL-10

#	Best Practice	Recommended Setting/Value	Priority	Source
8	Segment by `app.version` for release monitoring	Enables crash rate and performance comparison across releases	**Recommended**	MOBL-10

Priority Summary

Priority	Count	Description
Critical	36	Must implement -- failure to do so causes data loss, security risk, broken monitoring, or compliance violation
Recommended	39	Should implement -- significantly improves data quality, operational efficiency, or user experience
Optional	5	Nice to have -- advanced capabilities for mature mobile monitoring programs

Implementation Order

1. **First**: All Critical items in SDK Setup, Crash Reporting, and Privacy sections
2. **Second**: Critical items in User Action Instrumentation and Dashboards/Alerting
3. **Third**: All Recommended items
4. **Last**: Optional items for advanced use cases

References

- Dynatrace Mobile Monitoring Documentation
- iOS SDK Integration Guide
- Android SDK Integration Guide
- Session Replay for Mobile
- Mobile SDK Privacy Settings
